

# SIMATS ENGINEERING

## Department of Computer Science & Engineering

### ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

|                       |                                                                                                                                                                                                                                                 |                      |                                                                     |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|---------------------------------------------------------------------|
| <b>Course Code:</b>   | <b>CSA12</b>                                                                                                                                                                                                                                    | <b>Course Title:</b> | <b>Computer Architecture for Multicore Systems</b>                  |
| <b>CO Assessed:</b>   | <b>CO5</b>                                                                                                                                                                                                                                      | <b>Assessment:</b>   | <b>ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)</b> |
| <b>Weightage:</b>     | <b>30%</b>                                                                                                                                                                                                                                      | <b>Total Marks:</b>  | <b>25</b>                                                           |
| <b>CO5 Statement:</b> | <i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i> |                      |                                                                     |
| <b>Student Name:</b>  |                                                                                                                                                                                                                                                 | <b>Reg.No.:</b>      |                                                                     |
| <b>Department:</b>    |                                                                                                                                                                                                                                                 | <b>Date:</b>         |                                                                     |

#### ANNEXURE A

#### Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

#### Code of Conduct:

I V. Reddy Sreya (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

#### Signature of the student:

---

## MARKS ALLOCATION

|           | <b>Problem Understanding (20%)</b> | <b>Algorithm Design (30%)</b> | <b>Use of Control Structures (20%)</b> | <b>Pseudocode Structure (10%)</b> | <b>Completeness (20%)</b> |
|-----------|------------------------------------|-------------------------------|----------------------------------------|-----------------------------------|---------------------------|
| <b>Q1</b> |                                    |                               |                                        |                                   |                           |
| <b>Q2</b> |                                    |                               |                                        |                                   |                           |
| <b>Q3</b> |                                    |                               |                                        |                                   |                           |
| <b>Q4</b> |                                    |                               |                                        |                                   |                           |
| <b>Q5</b> |                                    |                               |                                        |                                   |                           |

## RUBRICS FOR CALCULATION:

| <b>Criteria</b>           | <b>Weightage</b> | <b>Level 4 – Exemplary</b>                                            | <b>Level 3 – Proficient</b>               | <b>Level 2 – Developing</b>  | <b>Level 1 – Beginning</b>      |
|---------------------------|------------------|-----------------------------------------------------------------------|-------------------------------------------|------------------------------|---------------------------------|
| Problem Understanding     | 20%              | Identifies all inputs, outputs, constraints accurately                | Minor missing elements                    | Partial understanding        | Incorrect interpretation        |
| Algorithm Design          | 30%              | Complete, correct, and logically sound solution                       | Minor logical gaps                        | Partially correct logic      | Incorrect approach              |
| Use of Control Structures | 20%              | Efficient and appropriate use of loops, conditions, functions/modules | Minor inefficiencies                      | Limited or basic usage       | Incorrect or missing constructs |
| Pseudocode Structure      | 10%              | Well-structured, clear, consistent format, easy to follow             | Mostly clear with minor issues            | Difficult to follow in parts | Poorly structured and unclear   |
| Completeness              | 20%              | Handles all cases; optimized approach                                 | Handles most cases; reasonable efficiency | Handles basic cases only     | Incomplete or inefficient       |

# 1. DMA - Based Data Transfer Algorithm

START

INPUT : I/O Device , Memory Address , Data Size

1 Initialize DMA Controller

2 Set DMA Source = I/O Device

3. Set DMA Destination = Memory Address.

4 Set Transfer Size = Data Size

5. Configure DMA Mode = Transfer Mode

6. Enable DMA Interrupt

7. Send DMA Request to I/O Device

8. WHILE transfer is not complete

DO

DMA Controller transfers data

CPU performs other tasks

END WHILE

9. When transfer is complete :

DMA Controller generates interrupt

10. CPU receives DMA interrupt

11. CPU executes DMA Interrupt Service Routine

12. Check transfer status

13. Clear DMA interrupt flag

14. IF transfer successful THEN

Report "DMA Transfer Completed"

ELSE

Report "DMA Transfer Error"

END IF

STOP

## 2. Priority - Based Vector Interrupt Handling

START

1. Initialize Interrupt Controller

2. Create priority levels for all devices

3. Assign interrupt vector to each device

4. WHILE system is running DO

IF interrupt request occurs  
THEN

Identify interrupting device

Determine priority of the interrupt

Select highest-priority pending interrupt

Save current CPU state

Get interrupt vector address

Jump to corresponding interrupt

Service Routine (ISR)



Execute ISR

Clear interrupt request

Restore CPU state

Return to interrupted program

END IF

END WHILE

STOP

### 3. Dynamic Selection of Memory-Mapped I/O and I/O-Mapped I/O

START

INPUT: latency Requirement, Bandwidth Requirement

1. Read latency requirement

2. Read bandwidth requirement

3. IF latency requirement is HIGH

AND

bandwidth requirement is

HIGH THEN

Select 'Memory - Mapped I/O

Allocate device registers

inside the memory address

space

ELSE IF latency requirement is low AND

bandwidth requirement  
is low THEN

Select I/O - Mapped I/O

Allocate separate I/O  
address space

ELSE

Compare system requirements

IF memory address access is  
Preferred THEN

Select Memory - Mapped I/O

ELSE

Select I/O - Mapped I/O

END IF

4. Configure selected I/O  
mechanism

5. Start data communication

STOP

4. Bus Arbitration for PCIe, USB and Thunderbolt

START

INPUT: Multiple Devices

1. Initialize bus controller

2. Register all connected devices.



3. WHILE communication is required

DO

Collect bus access requests

FOR each requesting device

DO

Determine device

Priority

Determine bandwidth

requirement

Determine urgency

END FOR

Select device with highest arbitration

Priority

Grant bus access to

Selected device

Allow selected device to  
transfer data

Monitor transfer

IF transfer is completed

THEN

Release bus

END IF

check for new requests

END WHILE

5. Manage interface-specific communication:

PCIe → high-speed device communication

USB → Peripheral communication

Thunderbolt → high-speed external devices

STOP

### 5. Hybrid I/O Using Polling and DMA Interrupts

START

INPUT: I/O Device, Data Size

1. Detect I/O device

2. Determine device speed

3. IF device speed is LOW THEN  
Use Polling

WHILE device is not ready

DO

check device status

END WHILE

Transfer data using polling

ELSE

Use DMA

Configure DMA Controller

Set source address

Set destination address

Set transfer size

WHILE DMA transfer is in progress

DO

CPU performs other tasks

END WHILE

END IF

4. Report transfer completion

STOP